

Project

Implementation of a Multi-Client Chat System with Lamport Clock Synchronization

Realized by:
Amir Ben Rejeb & Bahaeddine Hamrouni

Table of Contents:

1. Abstract
2. Introduction
3. Objectives
4. Technologies Used
5. System Design
    a. Architecture
    b. Components
    c. Communication Protocol
6. User Interface Design
7. Lamport Clock Integration
8. Implementation Details
9. Testing and Results

---

1. Abstract

This report presents a detailed study on the design, implementation, and evaluation of a distributed multi-client chat system using Java. The system integrates Lamport logical clocks to track the order of events across multiple clients, ensuring consistent event ordering in a distributed environment without relying on synchronized physical clocks. Screenshots and demo placeholders are included to illustrate system functionality.

2. Introduction

Distributed systems require consistent ordering of events to prevent conflicts and ensure reliable communication. Lamport clocks provide a logical mechanism to track causal relationships between events. This project demonstrates the integration of Lamport clocks into a multi-client chat application with private messaging.

3. Objectives

- Develop a client-server chat system in Java capable of handling multiple clients.
- Enable private messaging with real-time updates.
- Integrate Lamport clocks for logical synchronization.
- Implement a GUI to visualize messaging and clock updates.

4. Technologies Used

- Java 23 - TCP/IP Sockets for networking
- Swing for GUI
- Concurrent collections (ConcurrentHashMap, CopyOnWriteArrayList) for thread safety
- PrintWriter and BufferedReader for I/O streams



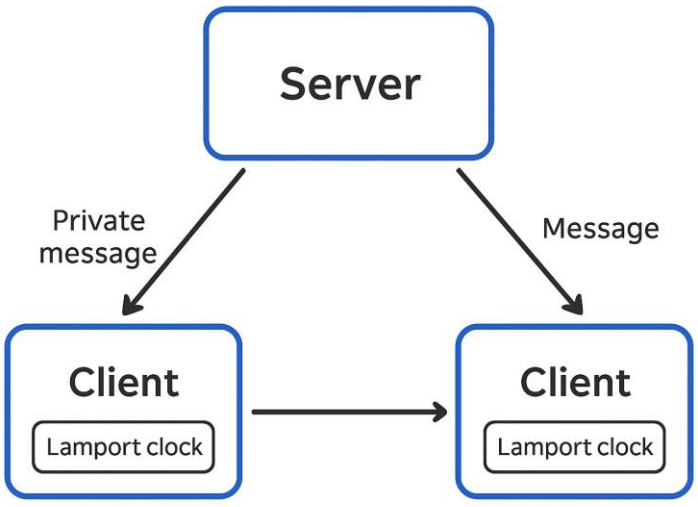


5. System Design:

5.1 Architecture:

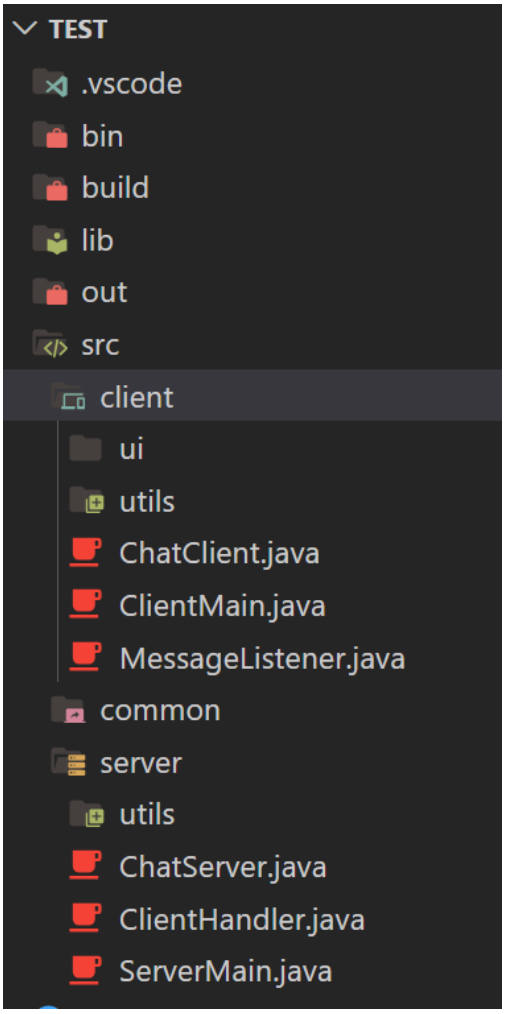
Centralized client-server model:

- Server handles client connections, user lists, and message routing
- Clients maintain Lamport clocks and provide GUI for messaging



5.2 Components:

The system is composed of 3 main parts Server , Client and Common folders .Each folder contains the program Classes following this arborescence :



- Arborescence of the Project

The following files are the most important :

- **ChatServer.java:** Accepts connections, manages clients, routes messages, and broadcasts user lists
- **ClientHandler.java:** Manages communication for individual clients
- **ChatClient.java:** Handles Lamport clock updates, sending/receiving messages, and GUI integration
- **MultiChatWindow.java:** Tabbed interface for private messaging, user list, input fields, and Lamport clock display

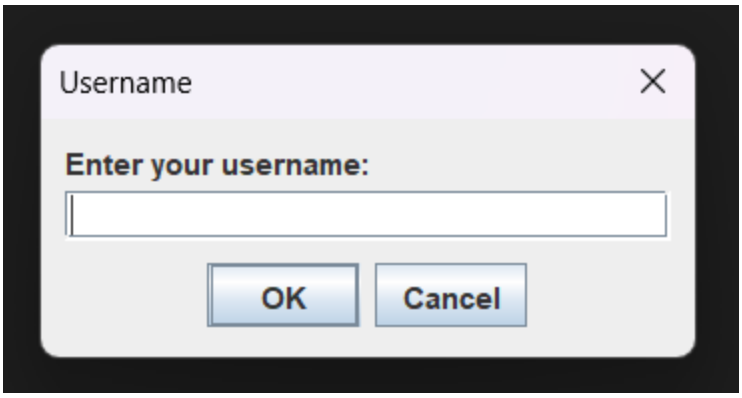
5.3 Communication Protocol:

- Messages sent in format: `PRIVATE:<sender>:<target>:<message>:<lamportClock>` - Server forwards messages to target client only
- USERLIST messages broadcasted when users join/leave

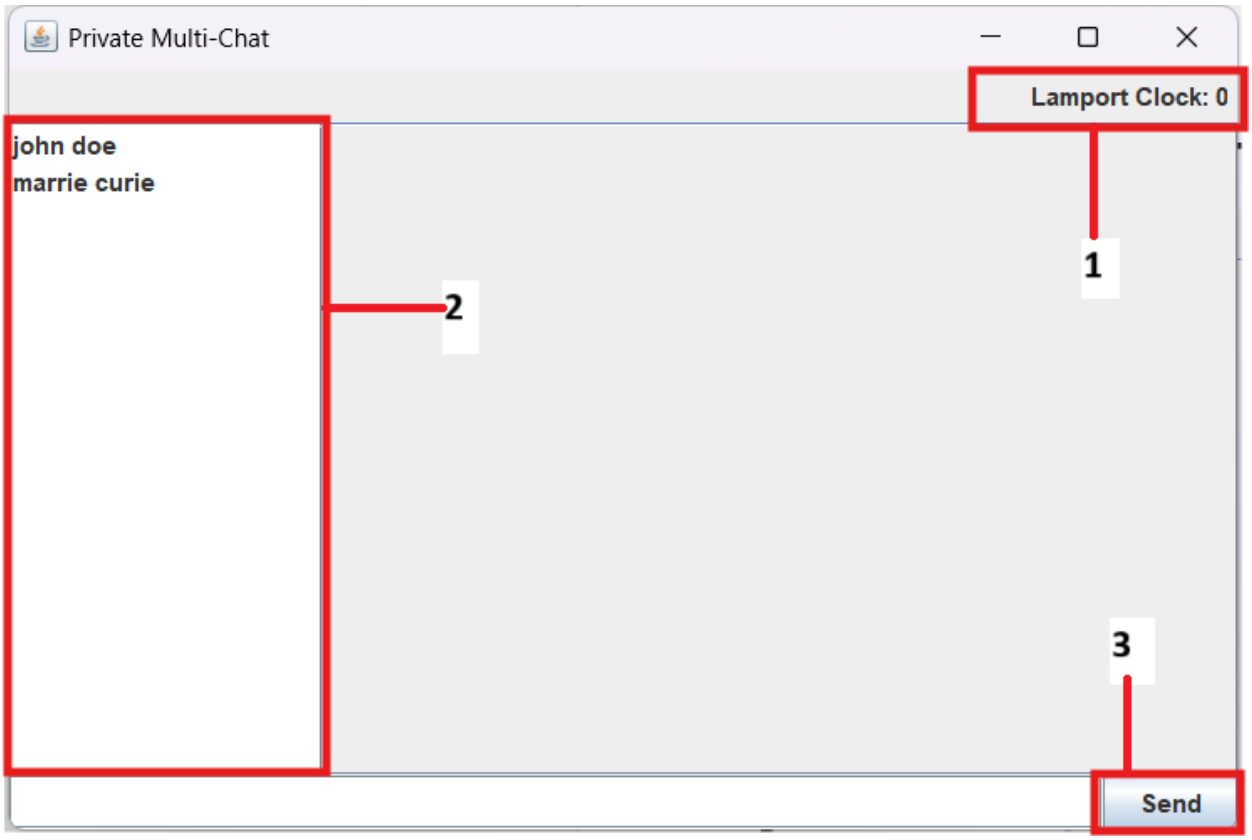
6. User Interface Design

A user interface was designed for sending and receiving messages and displaying Lamport clock ,it contains :

- Tabbed layout for private conversations
- User list for connected clients
- Input field and send button
- Lamport clock label for logical time
- Auto-scroll for latest messages



- Add new user form



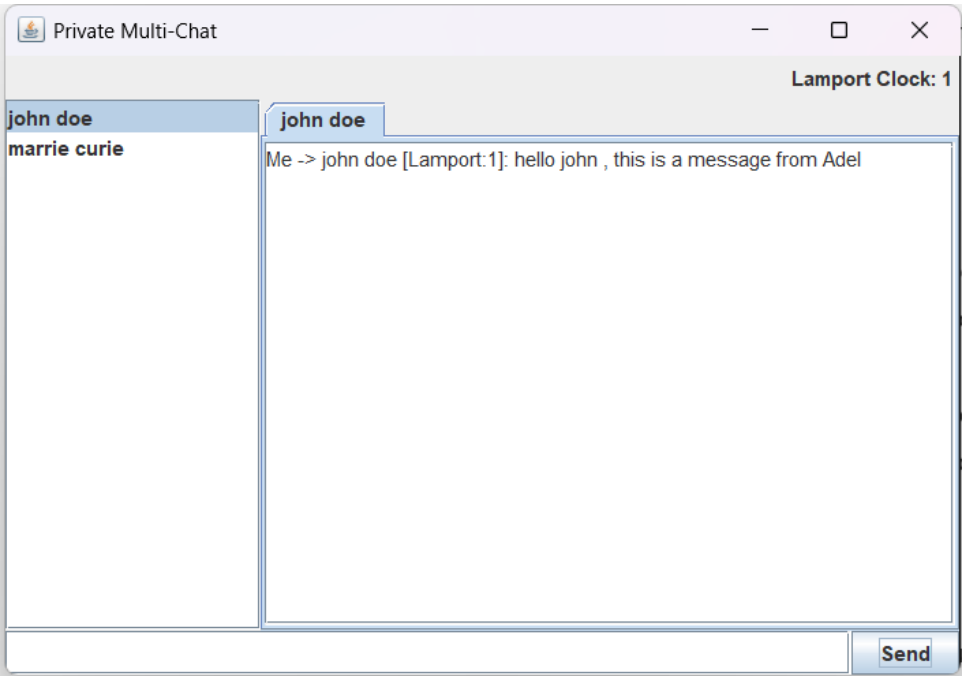
---

1. Lamport clock
2. Connected users list
3. Send message button

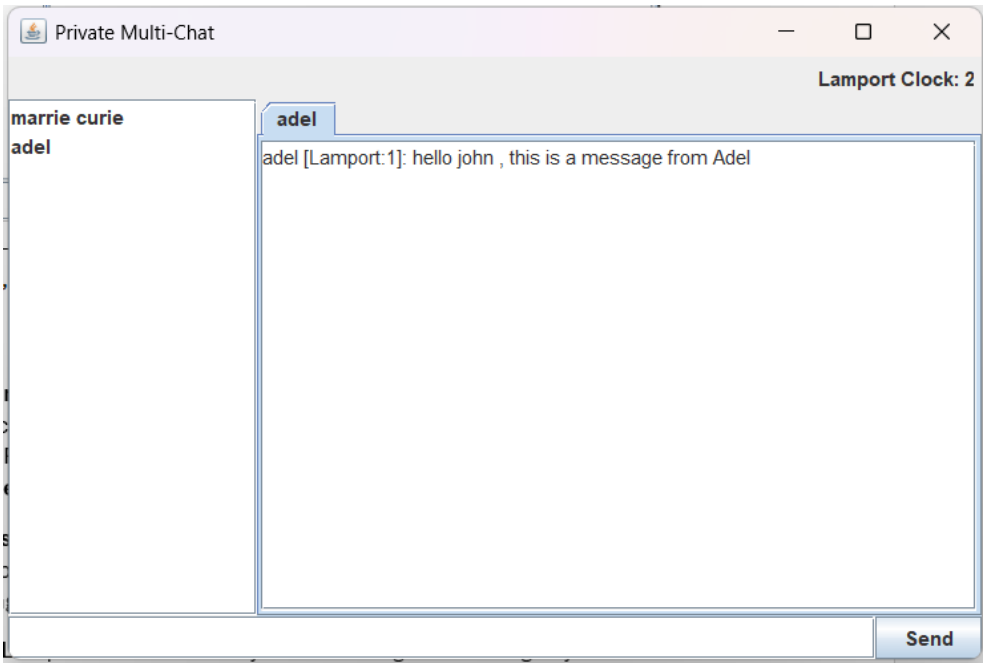
7. Lamport Clock Integration:

For synchronization Lamport algorithm was deployed , the following steps describe the workflow of the algorithm :

- Local Lamport clock maintained per client
- Increment on send, update on receive using `max(local, received)+1`
- Clock value displayed in GUI



- On message send, the Lamport clock increment for the sender

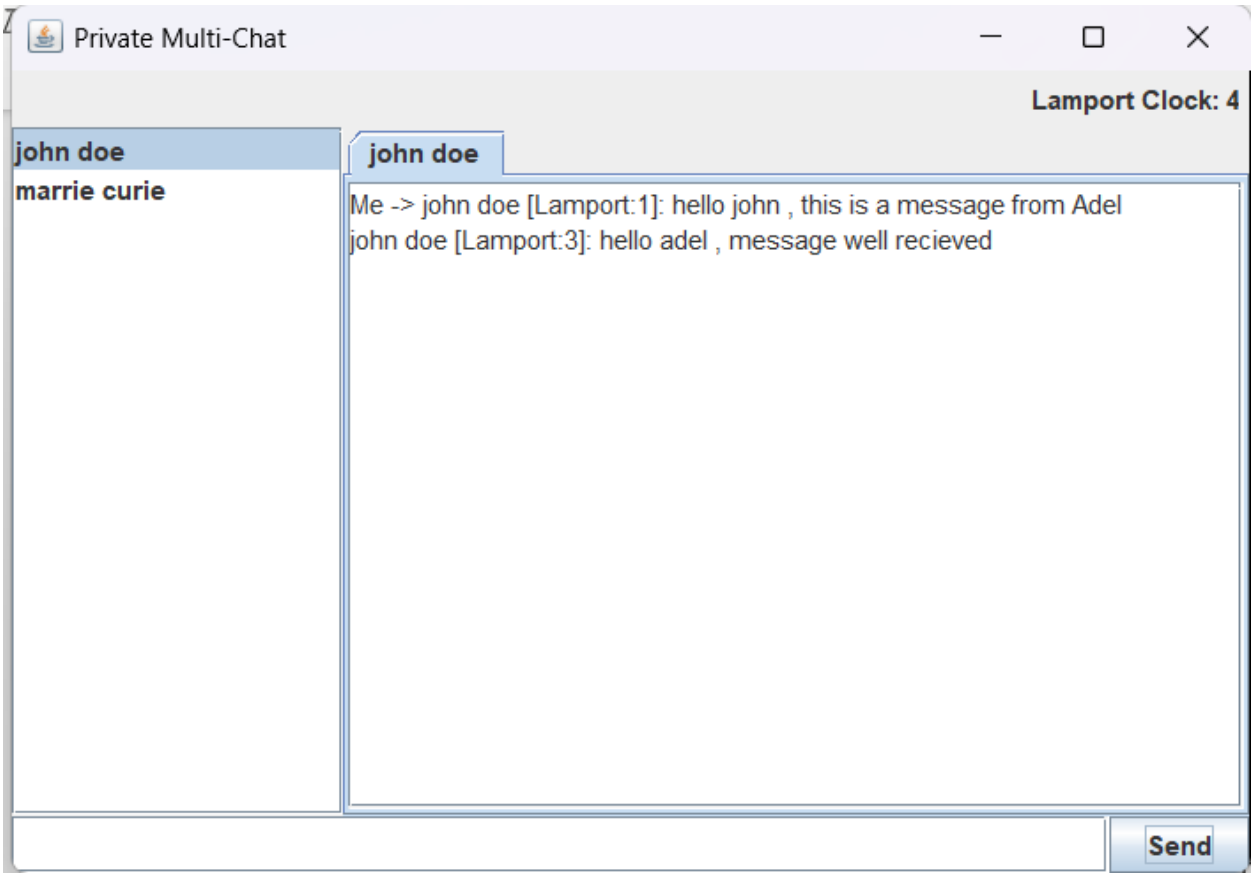


- Message Received in private tab
- Lamport clock updated for the receiver `max(local, received)+1`

8. Implementation Details

The system architecture and implementation incorporate several key design principles to ensure reliability, responsiveness, and correct message delivery:

- Thread-safe collections for client management
- Listener threads handle incoming messages asynchronously
- GUI updates executed in Swing thread for safety
- Private messaging ensures only selected recipient receives message



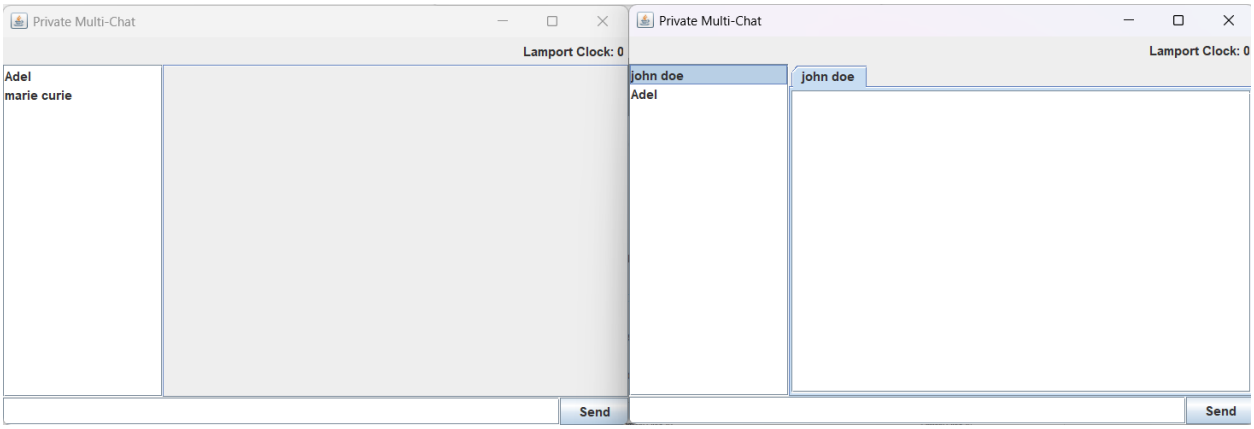
- Message recieved back with lamport clock [3]
- Lamport clock on client “Adel” updated to [4]

9. Testing and Results:

The following functionality and behaviors have been successfully tested :

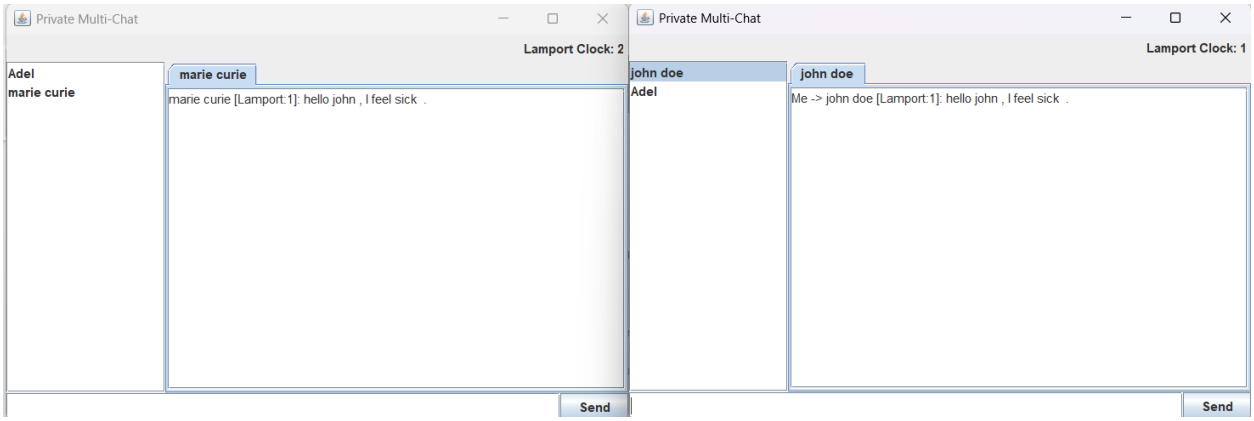
- Multiple clients connected simultaneously
- Verified private messaging and tab display
- Lamport clock increments correctly on send/receive
- Tested concurrent messaging to verify causal ordering

This following sequence demonstrates the workflow of the system containing 3 clients manages by the same server with Lamport clock algorithm for message synchronization



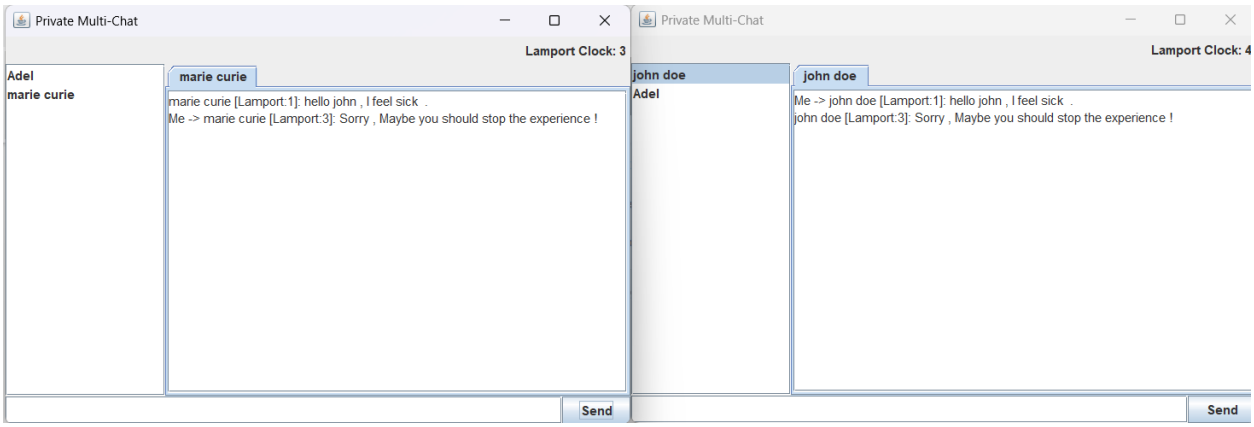
Scene 1

- System initialization with 3 clients [“Adel”,”John Doe”,”Marie Currie”].
- Lamport clock initialized “0”



Scene 2

- “Marie currie” initiate the system by sending a message and a timestamp with Lamport clock
- Message well recieved by “John Doe”



Scene 3

- Reply sent by “John Doe “with the new timestamp
- Both Lamport clocks are updated on Sending and on Receiving